Google Photos -> NAS / Immich

Public migration guide

A consolidated operational guide covering imports, shared-album reconstruction, remote downloads, recovery, storage, and lessons learned.

Compiled: 2026-10-14

Environment: UGREEN-class NAS, Docker, Immich, multiple source libraries

Privacy: generic labels and placeholders are used throughout this public edition.

PUBLIC EDITION	Review deployment-specific additions before publishing.

Contents

Use the bookmarks panel in a PDF viewer for fast navigation.

Google Photos → NAS / Immich migration 5
1. Executive summary 5
2. Sources used for this document 5
3. Related workstreams 5
4. System topology and paths 6
4.1 Logical architecture 6
4.2 Important host and container paths 6
4.3 Docker terminal container used for the migration 7
5. Remote access and terminal options 7
5.1 Preferred remote access model 7
5.2 Terminal choices 7
5.3 Chromium web terminal security 8
6. Google Photos export strategy 8
6.1 Three different kinds of export 8
6.2 Shared album ownership caveat 8
6.3 NAS-hosted Chromium downloader 8
6.4 Downloader lessons learned 9
6.5 rclone 9
7. Per-user media import with immich-go 9
7.1 Import policy 9
7.2 Correct shell command form 10
7.3 Dry-run behavior 10
7.4 Job-pause error 10
7.5 Logs and concurrent runs 10
7.6 Duplicate output from immich-go 11
8. API keys and permission model 11
8.1 Environment variable naming 11
8.2 Required capability model 11
8.3 Album members must be editors 12
8.4 Inspecting the actual member role 12
9. Reconciler design 12
9.1 plan is read-only 13
9.2 apply changes album membership and may upload 13
9.3 rollback is membership-only 13
9.4 Matching rules 14
9.5 Membership detection fix 14
9.6 Inventory cache and refresh modes 14
9.7 Forced external upload patch 15
9.8 Workspace artifact inventory 15

9.9 Representative migration scale . . . 16
10. Configuration patterns . . . 17
10.1 Five-user configuration skeleton . . . 17
10.2 Manifest mode for many ZIPs . . . 17
10.3 Name/path exactness . . . 18
11. Standard end-to-end runbook . . . 18
Step 0 — stop competing work and check space . . . 18
Step 1 — verify the container and inputs . . . 18
Step 2 — set keys for this terminal session . . . 19
Step 3 — build or reuse the plan . . . 19
Step 4 — inspect the plan before applying . . . 19
Step 5 — apply existing matches only . . . 19
Step 6 — apply with explicit external uploads . . . 20
Step 7 — monitor safely . . . 20
Step 8 — inspect the journal . . . 20
Step 9 — verify in Immich . . . 20
Step 10 — rollback only if necessary . . . 21
12. Recovery and debugging matrix . . . 21
12.1 “Plan contains ambiguous files” . . . 21
12.2 “Albums not found” even though the UI shows them . . . 21
12.3 “Reference path does not exist” . . . 21
12.4 “No permission” on PUT /albums/{id}/assets . . . 22
12.5 Asset appears in the UI but API verification shows zero . 22
12.6 immich-go returns code 0 but reconciler says matches_under_owner=0 . . . 22
12.7 Stuck after an inventory page . . . 23
12.8 Stuck after many [SKIP] already in album lines . . . 23
12.9 Safe resume after interruption . . . 23
12.10 Partial status . . . 23
12.11 Shell/tailing mistakes . . . 23
13. Storage and cleanup . . . 24
13.1 Why Immich admin and filesystem totals differed . . . 24
13.2 What consumed migration space . . . 24
13.3 Safe cleanup sequence . . . 24
13.4 Docker cleanup . . . 25
13.5 PostgreSQL and library safety . . . 25
14. Optional post-migration duplicate audit . . . 25
14.1 Exact duplicates versus visual similarity . . . 25
14.2 Safe database access pattern . . . 25
14.3 Cleanup policy . . . 26
15. Detailed chronological lessons learned . . . 26
Phase A — initial architecture . . . 26

Phase B — first import tests 26
Phase C — first reconciler behavior 26
Phase D — multi-user and album-member behavior 26
Phase E — large batches 27
Phase F — ambiguous/fallback/external cases 27
Phase G — storage pressure 27
Phase H — remaining Large shared album album 27
16. Recommended Large shared album procedure 27
16.1 Create the two temporary albums in Google Photos UI . . 27
16.2 Download directly to the NAS 28
16.3 Use two references for the same Immich album 28
16.4 Apply conservatively 28
17. Useful command appendix 28
Check process state 28
Check a journal summary 29
List upload failures 29
Find the last reconciler events 29
Test the Immich API 29
Search an album by name 29
Split a plan by album count 29
Validate the local copies before deployment 30
18. Things not to do 30
19. Final operational checklist 30

Google Photos → NAS / Immich migration

Consolidated operational notes, runbook, scripts, failure analysis, and lessons learned from the Google Photos migration work.

Compiled: 2026-10-14 Environment: UGREEN-class NAS, Docker, Immich, multiple source libraries Source libraries: User A through User E Server URL : `IMMICH_SERVER_URL` (replace with your private URL)

> Public-sharing notice: This copy uses generic labels and placeholders. Review any deployment-specific additions before publishing.

> This document is a historical and operational record. It is not proof of the current live NAS state. Before any destructive action, verify the live paths, active processes, current album counts, API permissions, and available storage.

1. Executive summary

The migration has two different jobs that must stay conceptually separate:

- Import media into each user's Immich library. This is done with immich-go from Google Takeout ZIPs. Each user's media belongs to that user.
- Reconstruct shared Google Photos albums in Immich. This is done by the custom google_album_reconciler.py script. It compares a Google Photos album "Download all" reference ZIP with the five users' imported Immich assets, then adds existing asset IDs to the canonical album. Media that none of the five users owns can optionally be uploaded under the album owner.

The reconciler deliberately does not edit PostgreSQL. It uses the Immich API for album membership and uses immich-go only for explicitly requested external uploads.

The overall migration reached the point where the configured album batches were processed, with a small number of unavoidable partial journals caused by video/duplicate verification behavior. The final unresolved type of work was the large Large shared album album: the source contained more items than the current Immich album. The recommended next step is to create two temporary Google Photos albums from the original Large shared album album, download both through the NAS-hosted Chromium browser, and reconcile the two ZIPs against the existing Immich album.

2. Sources used for this document

This guide combines:

- The full migration conversation, including the test run, production imports, reconciler development, album batches, failure recovery, storage investigation, and the final Large shared album question.
- The local migration workspace and the actual scripts/configuration files.
- Related Codex threads listed below.

The working notes contained secrets and deployment-specific identifiers. They are intentionally omitted here. Treat any key pasted into chat or shell history as exposed and rotate it.

3. Related workstreams

The migration was developed across several workstreams:

- Main reconciliation workflow: plans, applies, rollback, album ownership, permissions, storage, and final gap analysis.
- Remote download workflow: NAS-hosted Chromium, Playwright/CDP download monitoring, and archive verification.

- Reference-folder workflow: shared-album export limitations and Google Drive/rclone alternatives.
- Duplicate-audit workflow: read-only checksum analysis and safe Immich Trash cleanup.
- Cross-user attachment workflow: source-user permissions, album membership roles, and targeted inventory refresh.

4. System topology and paths

4.1 Logical architecture

```
Google Photos / Google Takeout
             |
             +-- per-user Takeout ZIPs
             |       |
             |       v
             |   immich-go
             |       |
             |       v
             |   Immich user libraries
             |
             +-- shared-album "Download all" reference ZIPs
                     |
                     v
               google_album_reconciler.py
                     |
          +----------+----------+
          |                     |
          v                     v
   Attach existing asset IDs   Upload missing external media
   using the source user key   under the album owner via immich-go
          |                     |
          +----------+----------+
                     v
             Canonical Immich album
```

4.2 Important host and container paths

These paths appeared in the working setup. They are not interchangeable:

Purpose	NAS host path	Reconciler-container path
Migration root	/HOST/MIGRATION_ROOT or /HOST/MIGRATION_ROOT depending on project	/srv/migration
Reference ZIPs	/HOST/MIGRATION_ROOT/reference	/srv/migration/reference
Reconciler application	/HOST/MIGRATION_ROOT/app	/srv/migration/app or /app
Plans, journals, SQLite inventory, staging	/HOST/MIGRATION_ROOT/reconciler-state	/srv/migration/reconciler-state
immich-go binary	/HOST/IMMICH_GO_DIRECTORY/immich-go	/immich-go/immich-go
Immich library, from docker inspect	/srv/immich/library	/data inside immich_server
Immich PostgreSQL data	/srv/immich/postgres	/var/lib/postgresql/data inside immich_postgres

The most common path mistake was using a NAS host path in a Python container. Once inside the reconciler container, use the mounted container path. For example, a ZIP at the host path /HOST/MIGRATION_ROOT/reference/test.zip is normally /srv/migration/reference/test.zip in the container.

4.3 Docker terminal container used for the migration

The working Compose project was:

```
services:
  album-reconciler:
    image: python:3.12-slim
    container_name: album-reconciler
    hostname: album-reconciler
    working_dir: /srv/migration
    network_mode: host
    stdin_open: true
    tty: true
    entrypoint: ["/bin/sh", "-c"]
    command: ["sleep infinity"]
    volumes:
      - /HOST/MIGRATION_ROOT:/srv/migration
      - /HOST/IMMICH_GO_DIRECTORY:/immich-go:ro
    restart: "no"
```

The container is intentionally just a terminal/runtime. It publishes no port. network_mode: host allows it to reach the internal Immich service at NAS_HOST:2283.

The checked-in workspace copy is docker/test-migration/docker-compose.yml.

5. Remote access and terminal options

5.1 Preferred remote access model

The initial recommendation was Tailscale:

- No router port forwarding for Immich, SSH, SMB, or UGOS.
- Encrypted WireGuard connections between authorized devices.
- A private 100.x.x.x address or MagicDNS name for the NAS.
- The Immich mobile app can use the private Tailscale URL while the phone is away from home.
- MFA, device authorization, ACLs/grants, and timely updates are still required.

Tailscale is not magic isolation: its coordination service can see device and connection metadata, although it does not receive the plaintext WireGuard traffic or private endpoint keys. Do not enable subnet routing or an exit node unless required.

UGREENlink/UGOS remote web access can be useful for opening the UGOS UI and Docker terminal, but a browser-friendly proxy URL such as https://NAS_PRIVATE_PROXY/photos should not automatically be assumed to be a valid Immich mobile API base URL. The mobile client needs the correct Immich API routing, stable HTTPS behavior, and authentication. Tailscale or a properly configured reverse proxy is safer and more predictable.

5.2 Terminal choices

When not on the home network, the practical choices were:

- Open the terminal for the running migration container in the UGOS Docker UI.
- Use a NAS-hosted Chromium web terminal, if the LinuxServer Chromium image is being used.
- Use a temporary Python or shell container with the migration folder mounted.
- Use SSH over Tailscale when host-level access is needed.

If /bin/bash is missing, use /bin/sh, ash, or a Python container. The reconciler and the main commands only require Python 3 and standard shell utilities. Do not expose Docker, SSH, SMB, or PostgreSQL ports directly to the Internet.

5.3 Chromium web terminal security

The LinuxServer Chromium container's web UI terminal can effectively control the browser container and, depending on mounts, the downloaded files. Anyone who can access that UI should be treated as having control of the browser session and mounted data. Keep the web UI behind UGREENlink/Tailscale/private access and never port-forward it publicly.

6. Google Photos export strategy

6.1 Three different kinds of export

These were often confused during the migration:

- Per-user Google Takeout: the normal source for importing a user's own library with immich-go.
- Google Photos album "Download all": the best reference for the complete visible contents of a shared album, including items uploaded by people outside the five migrating users.
- Google Drive + rclone: useful when Takeout archives have already been placed in Drive, but not a direct Google Photos shared-album API.

6.2 Shared album ownership caveat

An album owner's Google Takeout is not a guaranteed complete export of every media item visible in a shared album. It generally exports assets in the owner's library. A photo or video uploaded by another person may be visible in the shared album but absent from the owner's Takeout unless the owner saved a copy to their own library.

Therefore:

- Use the owner's Takeout with --sync-albums=true to create the canonical Immich album and import owner-owned media.
- Use a complete Google Photos "Download all" reference to discover the album's actual visible membership.
- Let the reconciler find the other five users' copies where possible.
- If a reference item belongs to nobody in Immich, explicitly upload it under the album owner only when that is the chosen policy.

Saving the whole shared album to the owner's library before Takeout can work, but it creates copies and may increase Google/NAS storage use. It is not the preferred method when a direct album download is available.

6.3 NAS-hosted Chromium downloader

The NAS browser project used a LinuxServer Chromium container similar to:

```
services:
  chromium:
    image: lscr.io/linuxserver/chromium:latest
    container_name: google-photos-browser
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=UTC
      - PIXELFLUX_WAYLAND=false
    volumes:
      - /HOST/BROWSER_CONFIG:/config
      - /HOST/MIGRATION_ROOT/reference:/downloads
    ports:
      - "3001:3001"
    shm_size: "1gb"
    restart: unless-stopped
```

In the browser:

- Open the Chromium web UI through the private remote path.
- Sign in to Google Photos.
- Open the album.
- Use Download all.
- Set browser downloads to /downloads.
- Confirm the ZIP appears in /HOST/MIGRATION_ROOT/reference on the NAS.

The traffic is Google → NAS; the laptop only displays the remote browser. The browser session is stored under the mounted /config, so protect that directory.

6.4 Downloader lessons learned

The Playwright downloader evolved to:

- Connect to the Chromium container through CDP.
- Observe Chromium Browser.downloadProgress events.
- Wait for Google Photos' "Preparing to download …" state.
- Locate album headers by exact visible data-title rather than assuming semantic headings.
- Scope the nearby "More options" button carefully because album titles are ordinary div elements.
- Keep a .zip.part file until the download and optional CRC verification finish.

If a completed-looking ZIP remains .zip.part, it may be undergoing CRC verification. VERIFY_ZIP_CRC=true is safer and verifies the archive before turnover. VERIFY_ZIP_CRC=false improves turnover speed but removes that verification step.

The browser's internal 8082 Selkies websocket port is not the normal web page port. Check the actual mapping with:

```
docker port google-photos-browser
```

If Chromium crashes with Aborted (core dumped) in the Wayland compositor, set PIXELFLUX_WAYLAND=false to force the X11 fallback and recreate the container.

6.5 rclone

rclone is useful for moving Takeout archives from Google Drive to the NAS without routing them through a laptop. The working container template is outputs/rclone-download.compose.yml.

It does not turn Google Photos shared albums into an rclone remote. It copies objects from Drive. For a complete shared album, the NAS browser's "Download all" route is the more reliable source.

7. Per-user media import with immich-go

7.1 Import policy

For the five migrating users, all imports used --sync-albums=true because each user may own albums that are shared with the others. The owner's import creates the album; the later reconciler adds media from the other user libraries.

Typical choices:

- --include-partner=false: do not import partner-library media automatically.
- --include-trashed=false: skip Google Photos trash.

- --pause-immich-jobs=false: avoid requiring admin job-control permissions and avoid pausing thumbnail generation.
- --dry-run: preview only; no media should be imported.
- --on-errors=continue: continue through individual errors and inspect the log afterward.

7.2 Correct shell command form

Inside the migration container:

```
export IMMICH_USER_A_API_KEY='REDACTED'

/immich-go/immich-go upload from-google-photos \
  --server="http://NAS_HOST:2283" \
  --api-key="$IMMICH_USER_A_API_KEY" \
  --include-partner=false \
  --include-trashed=false \
  --sync-albums=true \
  --pause-immich-jobs=false \
  /srv/migration/user-a/*.zip
```

For User B, replace the API variable and source directory. Do not paste Markdown links into the command. The correct line continuation is a single backslash at the end of the previous line, followed by --server; never write \--server.

The command failure below was caused by Markdown URL text and malformed continuations:

```
bash: --server=[http://NAS_HOST:2283](http://NAS_HOST:2283): No such file or directory
bash: --api-key=...: command not found
```

7.3 Dry-run behavior

--dry-run performs discovery and shows what would happen. It may show prospective tags, metadata actions, and album actions in the UI, but it should not import the media. It is not a cache that makes the subsequent live run unnecessary. Removing --dry-run starts a real import; immich-go may skip server duplicates based on its own server-side checks.

The dry-run output is complete when the progress reaches 100% and the process returns to the shell. A live run is complete when the command exits and the final summary/log is available; inspect errors even if most items were uploaded.

7.4 Job-pause error

A recurring error was:

```
PUT /api/jobs/thumbnailGeneration, 403 Forbidden
```

This came from immich-go trying to pause or resume Immich jobs without the required administrator permission. It does not mean all media failed. Use --pause-immich-jobs=false for the migration, or use an appropriately privileged admin key only if pausing jobs is intentionally required.

7.5 Logs and concurrent runs

immich-go logs were under:

```
/root/.cache/immich-go/immich-go_*.log
```

Example inspection:

```
ls -lt /root/.cache/immich-go/
tail -n 200 /root/.cache/immich-go/immich-go_YYYY-MM-DD_HH-MM-SS.log
grep -n -E 'ERR|ERROR|failed|403|Forbidden|duplicate|read/write|timeout' \
  /root/.cache/immich-go/immich-go_YYYY-MM-DD_HH-MM-SS.log
```

One migration run showed approximately 5,386 images and 404 videos; another showed 1,141 images and 76 videos. A small error count does not necessarily mean a bad import, but every error should be classified.

Do not start several heavy imports while Google downloads and Immich thumbnail/ML work are already consuming the NAS. Temporary 100% CPU is possible, but concurrent imports increase disk pressure and make API/ML delays harder to diagnose.

7.6 Duplicate output from immich-go

Lines such as:

```
server has duplicate
```

normally mean Immich already has an identical asset. This is usually a successful deduplication outcome, not a reason to keep uploading the same bytes. When the reconciler cannot verify an uploaded file after immich-go returns code 0, inspect the server for the checksum and album membership before retrying. Repeatedly retrying can create confusing staging and journal records even when the intended album is already correct.

8. API keys and permission model

8.1 Environment variable naming

The JSON config stores the name of an environment variable, not the API key itself:

```
{
  "name": "User A",
  "api_key_env": "IMMICH_USER_A_API_KEY"
}
```

Correct shell setup:

```
export IMMICH_USER_A_API_KEY='REDACTED'
export IMMICH_USER_B_API_KEY='REDACTED'
export IMMICH_USER_C_API_KEY='REDACTED'
export IMMICH_USER_D_API_KEY='REDACTED'
export IMMICH_USER_E_API_KEY='REDACTED'
```

An earlier failure used the actual key as api_key_env, producing an error like:

```
Environment variable REDACTED... is not set for User A
```

8.2 Required capability model

Exact permission names vary by Immich version, but the reconciler needs the equivalent of:

- Read the user’s assets.
- Read albums.
- Add and remove assets from albums (albumAsset.create, albumAsset.delete).
- Share/read assets where cross-user attachment is involved (asset.share or equivalent).
- Album sharing for the owner if the script automatically adds members (album.share or equivalent).

The key distinction is ownership:

- An existing User A asset is attached using User A's key.
- An existing User C asset is attached using User C's key.
- An external reference item is uploaded under the destination album owner, then attached using the owner's key.
- An album owner's admin status does not automatically make another user's asset attachable through the owner's key.

8.3 Album members must be editors

For a User A-owned album containing an User C asset, User C must be a member with editor permissions and the attachment request must use User C's key. A viewer can see the album but cannot add their own asset. The reconciler can add/promote configured members using:

```
"album_members": {
  "users": "all",
  "role": "editor"
}
```

or an explicit list:

```
"album_members": {
  "users": ["User B", "User E", "User D"],
  "role": "editor"
}
```

The owner is skipped as a duplicate member. New membership changes are journaled and can be removed by rollback.

8.4 Inspecting the actual member role

Immich's album response uses a nested user object on the versions encountered. Do not assume .userId exists at the top level:

```
curl -fsS \
  -H "x-api-key: $IMMICH_USER_A_API_KEY" \
  "$SERVER/api/albums/$ALBUM_ID" |
  jq --arg id "$USER_C_ID" '
    .albumUsers[]? |
    select((.userId // .user.id // .id) == $id) |
    {
      resolved_id: (.userId // .user.id // .id),
      role,
      user: .user
    }
  '
```

An empty result means the member is not present in the response, not necessarily that the API key is wrong. Confirm the album ID, current response, and user ID first.

9. Reconciler design

The main source is outputs/google_album_reconciler.py. The NAS-staged copy is docker/test-migration/app/google_album_reconciler.py. At the time this guide was written, the two files matched byte-for-byte and both passed Python compilation.

The companion documents are:

- google_album_reconciler.README.md

- google_album_reconciler.RUNBOOK.md
- SETUP.md

9.1 plan is read-only

The plan operation:

- Loads and validates the configuration.
- Confirms API keys resolve from the environment.
- Resolves each configured album under the configured owner.
- Checks reference paths.
- Loads or refreshes the SQLite asset inventory.
- Enumerates media inside each ZIP or reference directory.
- Computes a SHA-1 for each reference media entry.
- Checks whether that asset is already in the canonical album.
- Matches the reference bytes against all configured users.
- Writes a JSON plan and prints a summary.

It does not add album assets, upload media, edit PostgreSQL, or pause Immich jobs.

9.2 apply changes album membership and may upload

The apply operation:

- Validates the plan server and tool version.
- Refuses ambiguous matches.
- Refuses filename+size fallback matches unless --allow-fallback is explicit.
- Refuses external missing media unless --upload-missing is explicit.
- Optionally ensures configured album members are editors.
- Reads current album membership through metadata search.
- Groups existing assets by source user and attaches them using that source user’s key.
- Verifies each batch by querying the album again.
- Stages external media and matching Takeout sidecars.
- Runs immich-go upload from-folder under the album owner.
- Performs targeted post-upload inventory checks.
- Requires exactly one owner-owned asset with the reference SHA-1 before attaching it.
- Writes an incremental journal after successful batches.

9.3 rollback is membership-only

Rollback removes only the album memberships recorded by the selected journal and removes members added by that run. It does not:

- Delete assets from Immich.
- Delete assets uploaded under the album owner.
- Delete albums.

- Delete reference ZIPs.
- Delete staging files.

This is intentional: a rollback corrects album membership, not the media library.

9.4 Matching rules

The reconciler uses this order:

- Exact SHA-1 of the reference bytes versus Immich's asset checksum.
- Optional filename + size fallback if it is unique and the plan/config allow it.
- Already in album takes precedence when one of several same-byte candidates is already a member.
- Ambiguous means more than one possible candidate; apply refuses to guess.
- External missing means no configured user owns an exact match; upload is opt-in.

The filename+size fallback is not proof that bytes are identical. It is useful when Google Takeout and Google Photos browser downloads produce different bytes for the same logical asset, but it can select the wrong duplicate when filenames are reused. The safest production policy used here was:

```
"match": {
  "allow_filename_size": false
}
```

If fallback matches are present, do not use --allow-fallback unless every match has been reviewed. When the desired policy is a fresh copy, mark or construct the plan for an explicit external upload instead of allowing fallback.

9.5 Membership detection fix

An important Immich API behavior was discovered: GET /api/albums/{id} returned album metadata and assetCount but did not reliably include the full asset list. The first reconciler implementation therefore reported assets as absent even when they were already in the album.

The script now enumerates membership through:

```
POST /api/search/metadata
{
  "albumIds": ["ALBUM_ID"],
  "withArchived": true,
  "withDeleted": false,
  "withExif": true,
  "withPeople": false,
  "withStacked": true,
  "order": "asc"
}
```

This change explains why later plans showed large already_in_album counts instead of trying to attach everything again.

9.6 Inventory cache and refresh modes

The SQLite inventory lives at:

```
/srv/migration/reconciler-state/inventory.sqlite3
```

It indexes user name, asset ID, checksum, normalized filename, file size, dates, and type.

Planning options:

```
# Use the existing cache if present.
python3 /srv/migration/app/google_album_reconciler.py plan \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --output /srv/migration/reconciler-state/plan.json

# Fetch all configured users again.
python3 /srv/migration/app/google_album_reconciler.py plan \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --output /srv/migration/reconciler-state/plan.json \
  --refresh-inventory

# Archive the current SQLite cache and rebuild it.
python3 /srv/migration/app/google_album_reconciler.py plan \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --output /srv/migration/reconciler-state/plan.json \
  --rebuild-inventory
```

The first inventory build can appear much faster because it creates a new SQLite database sequentially. Later full refreshes can spend substantial time deleting/replacing rows, writing indexes, committing, and waiting on the database filesystem. A log line such as received asset page=63 ... next_page=None means API pagination has ended; the following cache write can still be slow.

Apply supports:

- targeted: recommended; refreshes only the album owner's assets created after the relevant window and falls back to exact checksum queries.
- album_owner: refreshes the complete owner inventory.
- all: refreshes every configured user's complete inventory.

The production configs used "apply_inventory_refresh": "targeted" to avoid rescanning tens of thousands of assets between albums. --refresh-all-users overrides the config for a run.

9.7 Forced external upload patch

The current local script includes a force_external_upload plan-item flag. Normally apply rechecks a stale external item against the owner inventory before uploading, because a previous run might have imported it in the meantime. An item marked force_external_upload deliberately bypasses that reclassification and remains an upload candidate.

This is useful when a same-byte match exists elsewhere but policy requires a fresh copy under the album owner. It is an exception, not the default. The same script must be deployed to the NAS before using plans that contain this flag.

9.8 Workspace artifact inventory

The migration workspace contains these directly useful artifacts:

- outputs/google_album_reconciler.py: source/reference reconciler. Current tool version is 6.
- docker/test-migration/app/google_album_reconciler.py: NAS-staged copy. It matched the output copy when this guide was compiled.
- outputs/google_album_reconciler.README.md: concise design and safety documentation.
- outputs/google_album_reconciler.RUNBOOK.md: operational runbook.
- docker/migration/SETUP.md: first single-album test setup.
- outputs/google_album_reconciler.config.example.json: single/few-album configuration template.
- outputs/google_album_reconciler.multi.config.example.json: manifest-based multi-album template.
- outputs/google_album_config.json: early Shared Album Test example.
- docker/test-migration/app/google_album_reconciler.config.json: test/target configuration; review it before production use.

- docker/test-migration/app/google_album_reconciler.remaining.config.json: remaining-album batch configuration.
- docker/test-migration/app/google_album_reconciler.new-albums.config.json: later new-album batch configuration.
- outputs/immich-go-production.compose.yml: placeholder Compose project for a separate immich-go terminal.
- outputs/rclone-download.compose.yml: rclone terminal project for Google Drive-to-NAS copies.

The browser downloader/Playwright project was developed in the related remote-download thread rather than in this migration workspace. Its deployment bundle was recorded there as google-photos-playwright.zip under the 2026-09-28 now-x20 workspace.

9.9 Representative migration scale

These figures were observed during the migration and are useful for planning capacity, but they are historical:

Album or import	Approximate observed size/count	Operational lesson
User A import test	5,386 images + 404 videos, about 66.2 GiB	Large import produces long thumbnail/metadata work and many log lines.
User B import test	1,141 images + 76 videos, about 2.2 GiB	Small error counts require log classification, not immediate rollback.
Shared Album A	507 reference media; 487 already in album, 19 exact matches, 1 external	Good example of the reconciler's three-way outcome.
Large travel album A	1,788 items; 776 uploads; about 10.67 GiB total	Large external staging makes cleanup important.
Shared Album B	1,368 items; 1,017 uploads; about 9.54 GiB total	Video verification can leave a small partial journal even when most work succeeds.
Shared Album C	1,547 items; 313 uploads; about 14.09 GiB total	One unusual IMG.MP4 required a direct retry.
Travel album B	303 items; 272 uploads; about 3.90 GiB total	Good candidate for a single targeted batch after ambiguity resolution.
Travel album C	1,813 items; 776 uploads; about 8.49 GiB total	Very large plan; split implementation if storage is tight.
Travel album D	999 items; 367 uploads; about 6.06 GiB total	Batch only after large staging is removed.
Travel album E	1,343 items; 71 uploads; about 11.15 GiB total	Large total media does not necessarily mean large external staging.
Large shared album	the source count Google Photos items vs the current server count in Immich	Final known gap; use two temporary reference albums.

The migration therefore used a "largest but manageable, verify, clean, continue" strategy rather than starting every album at once.

10. Configuration patterns

10.1 Five-user configuration skeleton

Use placeholders and keep keys out of the file:

```
{
  "server": "http://NAS_HOST:2283",
  "state_dir": "/srv/migration/reconciler-state",
  "immich_go_path": "/immich-go/immich-go",
  "immich_go_pause_immich_jobs": false,
  "immich_go_concurrent_tasks": 1,
  "immich_go_client_timeout": "20m",
  "immich_go_on_errors": "continue",
  "apply_inventory_refresh": "targeted",
  "page_size": 1000,
  "batch_size": 100,
  "plan_log_interval": 250,
  "timeout_seconds": 60,
  "retries": 4,
  "match": {
    "allow_filename_size": false
  },
  "album_members": {
    "users": "all",
    "role": "editor"
  },
  "users": [
    { "name": "User A", "api_key_env": "IMMICH_USER_A_API_KEY" },
    { "name": "User B", "api_key_env": "IMMICH_USER_B_API_KEY" },
    { "name": "User C", "api_key_env": "IMMICH_USER_C_API_KEY" },
    { "name": "User D", "api_key_env": "IMMICH_USER_D_API_KEY" },
    { "name": "User E", "api_key_env": "IMMICH_USER_E_API_KEY" }
  ],
  "albums": [
    {
      "name": "Exact Immich album name",
      "owner": "User C",
      "reference": [
        "/srv/migration/reference/exact-reference-file.zip"
      ]
    }
  ]
}
```

10.2 Manifest mode for many ZIPs

The multi-album example is outputs/google_album_reconciler.multi.config.example.json.

Instead of a long albums array:

```
{
  "reference_directory": "/srv/migration/reference",
  "reference_manifest": "/srv/migration/reference/albums.json"
}
```

The manifest maps each ZIP to an exact Immich album and owner:

```
{
  "albums": [
    {
      "zip": "Shared Album A.zip",
      "name": "Shared Album A",
      "owner": "User C"
    }
  ]
}
```

The manifest mode intentionally refuses unlisted ZIPs and duplicate listings. This prevents a filename alone from silently inferring the wrong owner.

10.3 Name/path exactness

The following caused planning failures:

- A config path omitted .zip.
- A token in the filename was mistyped (...3lcw versus ...3lcw).
- The actual file had a browser-generated suffix such as 479 items · SharedMore options.
- A download was still .zip.part.
- An Immich album had a trailing space, for example Travel album R .
- Unicode punctuation or visually similar spacing differed between Google Photos, ZIP folder name, config, and Immich.

Always inspect exact bytes and shell quoting:

```
ls -lb /srv/migration/reference | grep '2024.10.03'
test -f '/srv/migration/reference/exact name.zip' && echo 'reference ZIP found'

curl -fsS \
  -H "x-api-key: $IMMICH_USER_C_API_KEY" \
  "$SERVER/api/albums" |
  jq -r '.[] | [.albumName, .id, .ownerId] | @tsv'
```

For visible whitespace:

```
curl -fsS \
  -H "x-api-key: $IMMICH_USER_C_API_KEY" \
  "$SERVER/api/albums" |
  jq -r '
    .[] |
    select(.albumName | test("Shared Album"; "i")) |
    "name=\((.albumName | @json)) length=\(.albumName | length) id=\(.id)"
  '
```

11. Standard end-to-end runbook

Step 0 — stop competing work and check space

```
ps aux | grep -E 'google_album_reconciler|immich-go' | grep -v grep
df -h /srv/migration
df -h /srv/immich/library
```

Do not rebuild the inventory while another reconciler is writing it. Do not run a second large import just because the first terminal appears quiet.

Step 1 — verify the container and inputs

```
ls -lh /srv/migration/reference/
ls -l /srv/migration/app/google_album_reconciler.py
ls -l /immich-go/immich-go
python3 --version
```

Check a ZIP before scanning it:

```
unzip -t '/srv/migration/reference/album.zip'
```

Step 2 — set keys for this terminal session

```
export IMMICH_USER_A_API_KEY='REDACTED'
export IMMICH_USER_B_API_KEY='REDACTED'
export IMMICH_USER_C_API_KEY='REDACTED'
export IMMICH_USER_D_API_KEY='REDACTED'
export IMMICH_USER_E_API_KEY='REDACTED'
```

Check identity without printing the key:

```
SERVER='http://NAS_HOST:2283'
curl -fsS -H "x-api-key: $IMMICH_USER_C_API_KEY" \
  "$SERVER/api/users/me" | jq '{id,email,name}'
```

Step 3 — build or reuse the plan

Use cached inventory when it is known current:

```
python3 /srv/migration/app/google_album_reconciler.py plan \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --output /srv/migration/reconciler-state/plan-YYYY-MM-DD.json
```

Use --refresh-inventory only when imports changed or the cache is not trustworthy. Use --rebuild-inventory when the SQLite cache is corrupted or intentionally being rebuilt; it archives the old cache rather than deleting it blindly.

Planning may write a partial JSON even when configuration validation fails. That JSON is useful for diagnosis but is not safe to apply while it contains validation_errors.

Step 4 — inspect the plan before applying

```
jq '.validation_errors // null' /srv/migration/reconciler-state/plan-YYYY-MM-DD.json
jq -r '
  .albums[] |
  [.name, .owner,
   (.counts.reference_media|tostring),
   (.counts.already_in_album|tostring),
   (.counts.matched_exact|tostring),
   (.counts.external_missing|tostring),
   (.counts.ambiguous|tostring)] | @tsv
' /srv/migration/reconciler-state/plan-YYYY-MM-DD.json
```

Required decisions:

- ambiguous must be zero, unless the item is removed/resolved in a new plan.
- Review every external_missing item before allowing owner upload.
- Treat filename+size fallback as a manual review list.
- Confirm each album’s owner and exact Immich album ID.

Step 5 — apply existing matches only

```
python3 /srv/migration/app/google_album_reconciler.py apply \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --plan /srv/migration/reconciler-state/plan-YYYY-MM-DD.json \
  --journal /srv/migration/reconciler-state/apply-YYYY-MM-DD.json \
  --yes
```

Step 6 — apply with explicit external uploads

```
python3 /srv/migration/app/google_album_reconciler.py apply \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --plan /srv/migration/reconciler-state/plan-YYYY-MM-DD.json \
  --journal /srv/migration/reconciler-state/apply-YYYY-MM-DD.json \
  --upload-missing \
  --yes
```

Do not reuse an existing journal filename unless deliberately using --overwrite-journal. A new journal gives a clean audit trail and avoids confusing old events with a retry.

Step 7 — monitor safely

For a foreground process, open another terminal and inspect:

```
ps aux | grep -E 'google_album_reconciler|immich-go' | grep -v grep
tail -f /srv/migration/reconciler-state/apply-YYYY-MM-DD.log
```

tail -f only follows output. Pressing Ctrl-C in the terminal running tail stops tail, not the producer process. Pressing Ctrl-C in the foreground reconciler terminal interrupts the reconciler and can leave a partial journal.

For a detached run:

```
nohup python3 /srv/migration/app/google_album_reconciler.py apply \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --plan /srv/migration/reconciler-state/plan-YYYY-MM-DD.json \
  --journal /srv/migration/reconciler-state/apply-YYYY-MM-DD.json \
  --upload-missing \
  --yes \
  > /srv/migration/reconciler-state/apply-YYYY-MM-DD.log 2>&1 &
echo $!
```

If nohup reports Exit 2, inspect the log; it means the process ended with a command/reconciler error, not that it is still running.

Step 8 — inspect the journal

```
jq '{status, error, warning, completed_at, failed_at}' \
  /srv/migration/reconciler-state/apply-YYYY-MM-DD.json

jq -r '
  .albums[] |
  "\(.name) | changes=\(.changes|length) | uploads=\(.uploads|length) |
failures=\(.upload_failures|length) | events=\(.events|length)"
' /srv/migration/reconciler-state/apply-YYYY-MM-DD.json
```

A successful complete run ends with:

```
Status: completed
```

Status: partial means either the process stopped/failed or one or more external uploads were not verified. It does not mean all changes were undone. Inspect the journal before retrying.

Step 9 — verify in Immich

For each album:

- Compare the Immich album count with the reference count.
- Inspect a mixture of images, videos, motion/live-photo pairs, and external-contributor items.
- Verify dates, locations, thumbnails, and playback.

- Verify source ownership in each user timeline.
- Verify all intended members can view the album.
- Search for duplicate uploads before deleting staging/reference data.

Step 10 — rollback only if necessary

```
python3 /srv/migration/app/google_album_reconciler.py rollback \
  --config /srv/migration/app/google_album_reconciler.config.json \
  --journal /srv/migration/reconciler-state/apply-YYYY-MM-DD.json \
  --yes
```

Rollback removes recorded memberships only. It does not remove photos uploaded under the album owner, so cleanup of unwanted uploaded media must be handled separately in Immich.

12. Recovery and debugging matrix

12.1 “Plan contains ambiguous files”

Cause: more than one Immich candidate has the same exact checksum, or the same fallback filename+size.

Action:

- Do not use --allow-fallback automatically.
- Inspect the plan item’s candidate user/asset IDs.
- Decide which existing asset is canonical, or deliberately force an external upload.
- Create a new resolved plan; do not edit the original without preserving a copy.
- Apply only after ambiguous: 0 for the selected album(s).

12.2 “Albums not found” even though the UI shows them

Likely causes:

- Trailing/leading whitespace in the Immich album name.
- Unicode normalization or visually similar punctuation.
- Wrong owner in config.
- The API key cannot see the album.
- The album was created under a different user by an earlier import.

Use /api/albums with the configured owner’s key, compare JSON-escaped names and lengths, and use an explicit album id in the config when names are problematic.

12.3 “Reference path does not exist”

Likely causes:

- Missing .zip extension.
- Typo in a generated token.
- Actual file name contains a suffix added by Google Photos/browser.
- File is still .zip.part.
- Host path was used inside the container.

Use ls -lb, quote the entire path, and update the config. Planning now preserves a partial plan with validation errors, but apply refuses it until fixed.

12.4 “No permission” on PUT /albums/{id}/assets

This is the most important cross-user failure mode.

Verify:

- The asset’s ownerId matches the source user key.
- The source user is an album editor.
- The request uses the source user’s key, not only the album owner’s key.
- The album ID is correct.
- The API key identity is what you expect.
- The script version includes member role handling and nested .user.id parsing.

An album owned by User A can contain an User C asset, but the attachment should be made via User C’s key after User C has editor membership. The log line “Adding 2 assets from User E” means the assets were found in User E’s inventory; it does not mean they are being uploaded as User E.

12.5 Asset appears in the UI but API verification shows zero

Possible causes:

- The queried endpoint is not filtering the album correctly.
- The API response structure is being parsed incorrectly.
- The wrong album ID or API key was used.
- Immich background processing has not made the asset searchable yet.

Use the same metadata-search endpoint used by the script and inspect raw JSON. Do not infer membership from GET /api/albums/{id} alone.

12.6 immich-go returns code 0 but reconciler says matches_under_owner=0

This occurred for several videos and one image. The most likely explanations were:

- Immich recognized the upload as a duplicate and did not create a new owner asset.
- Metadata/thumbnail processing delayed visibility.
- The upload log’s “server has duplicate” entry referred to an asset already present elsewhere.
- The owner inventory refresh window did not include the existing asset.

Actions:

- Inspect the immich-go log for the staged file name and server has duplicate.
- Search Immich by checksum or exact asset ID.
- Check whether the asset is already in the desired album.
- If yes, leave it; do not retry blindly.
- If the policy truly requires a new owner copy, create a plan item with force_external_upload and use the current script version, understanding that Immich may still deduplicate identical bytes.
- If the item is a video or unusual codec, test a manual immich-go upload from-folder for that single file and inspect the result.

12.7 Stuck after an inventory page

A line like:

```
[INVENTORY] received asset page=63 ... next_page=None
```

means HTTP pagination finished. The subsequent delay may be SQLite cache replacement/indexing, not a network hang. Do not immediately delete the SQLite file. Check CPU, disk I/O, and the process state first.

The targeted refresh mode was added specifically to avoid a full inventory refresh between every album.

12.8 Stuck after many [SKIP] already in album lines

This is normally progress. The script still has to finish the item list, flush journal events, process any external candidates, verify album membership, and write the final status. Check:

```
ps -o pid,etime,time,%cpu,%mem,stat,cmd -p "$PID"
tail -n 50 /srv/migration/reconciler-state/apply-YYYY-MM-DD.log
```

If the process is alive and CPU/time continues to advance, wait. If it is gone, inspect the journal and log.

12.9 Safe resume after interruption

The journal is incremental and membership checks are idempotent enough for a retry, but it is not a magic cursor. Use:

- The same reviewed plan.
- A new journal path.
- The same config and API keys.
- No unnecessary full inventory refresh.

Existing album membership is skipped. Do not use a stale plan after a major import unless the apply targeted refresh policy is enabled and the result is reviewed.

12.10 Partial status

A partial journal may contain many successful changes. Extract failures:

```
jq -r '
  .albums[] as $album |
  $album.upload_failures[]? |
  "\($album.name) | \(.reference) | owner=\(.user) | matches=\(.matches_under_owner) | immich-
go=\(.immich_go_returncode) | staged=\(.staged_path)"
' /srv/migration/reconciler-state/apply-YYYY-MM-DD.json
```

Retry only the failed album or failed item when possible. Do not re-run a large multi-album plan if one album has two failed videos.

12.11 Shell/tailing mistakes

- tail -f can be stopped with Ctrl-C without stopping the producer.
- Ctrl-C in the producer terminal interrupts the producer.
- nohup ... & ending with exit code 2 means inspect the log; it is not a successful background run.
- Keep shell variables such as SERVER, ALBUM_ID, and ASSET_ID visible and validate them with echo before issuing a write.
- Quote paths containing spaces, apostrophes, emoji, Chinese characters, or trailing spaces.

13. Storage and cleanup

13.1 Why Immich admin and filesystem totals differed

Observed measurements included:

```
df -h /srv/migration
1.8T filesystem, approximately 44G free

du -xhd1 /srv/migration
approximately 128G visible under /srv/migration

du -sh /srv/immich/library
approximately 1.6T

Immich admin UI
approximately 1.2 TiB
```

The apparent mismatch is expected because /srv/migration is not the Immich library mount. The Immich library was /srv/immich/library, and it includes more than the admin media-total number may show:

- Original assets.
- Encoded/transcoded video variants.
- Thumbnails and previews.
- Sidecars and metadata.
- Trash/deleted assets that have not been permanently removed.
- Filesystem allocation, snapshots, and other NAS overhead.

The reconciliation staging/reference files were additional space outside the Immich library.

13.2 What consumed migration space

Observed high-usage locations included:

```
/srv/migration/reference                        about 128G
/srv/migration/reference/.playwright-tmp        about 122G in one snapshot
/srv/migration/reconciler-state                 about 109M in a later snapshot
/srv/migration/reconciler-state/inventory.sqlite3 about 76M
/srv/migration/reconciler-state/external-upload-staging
                                               up to about 141G in an earlier snapshot
```

Temporary external-upload staging is the most important reconciler cleanup target after verification. It can contain extracted media and sidecars for every external candidate, including files that Immich already deduplicated.

13.3 Safe cleanup sequence

Only after the corresponding albums are verified:

- Keep the final plan and journal until acceptance/backup.
- Remove only reference ZIPs that are no longer needed.
- Remove stale .zip.part and .playwright-tmp files after confirming no download is active.
- Remove reconciler external staging directories for completed/accepted albums.
- Keep the active SQLite inventory if future plans depend on it.
- Keep a backup of the final journal and plan off the NAS.
- Use Immich’s own Trash/UI for unwanted media; do not delete library files manually.

Inspect first:

```
df -h /srv/migration
du -xhd1 /srv/migration 2>/dev/null | sort -h
du -xhd2 /srv/migration 2>/dev/null | sort -h | tail -40
du -sh /srv/migration/reference /srv/migration/reconciler-state 2>/dev/null
du -sh /srv/migration/reconciler-state/external-upload-staging 2>/dev/null
```

For the Immich library:

```
du -sh /srv/immich/library
```

This can take a long time on a large NAS. Do not interrupt a live Immich write or delete based only on a partial du result.

13.4 Docker cleanup

docker system df -v is the first read-only check. Do not run docker system prune -a or volume pruning blindly on a NAS running Immich; unused-looking images/volumes may be needed for rollback or databases. Inspect containers, images, and volumes first.

13.5 PostgreSQL and library safety

The Immich data paths were:

```
/srv/immich/library
/srv/immich/postgres
```

Do not edit PostgreSQL directly to reconcile albums. If database maintenance is required, back up first and use Immich-supported procedures. The reconciler's API-only design avoids corrupting metadata relationships.

14. Optional post-migration duplicate audit

The related duplicate-audit threads developed a read-only cross-user scan based on Immich PostgreSQL data and SHA-1 checksums.

14.1 Exact duplicates versus visual similarity

- SHA-1 duplicate groups indicate byte-identical files.
- They do not identify visually similar images with different bytes.
- CLIP/embedding similarity is a separate, more expensive analysis and can load CPU/DB.

One audit reported approximately 7,055 duplicate checksum groups, 8,164 redundant copies, and about 13.56 GiB potentially reclaimable. Treat those numbers as historical, not current.

14.2 Safe database access pattern

The PostgreSQL container did not publish a host port. A temporary Python container can use the Immich Docker network:

```
docker run --rm -it \
  --network immich_default \
  -v "$PWD:/work" \
  python:3.13-slim sh
```

Inside it:

```
pip install --no-cache-dir 'psycopg[binary]>=3.2,<4'
```

Use immich_postgres as the host, not 127.0.0.1 from the temporary container. Do not expose PostgreSQL to the LAN or Internet.

14.3 Cleanup policy

The safest sequence is:

- Generate a read-only report.
- Review every candidate and keep one owner/canonical asset.
- Move candidates to Immich Trash using the correct owner API permissions, if that workflow has been explicitly approved.
- Verify the candidates are in Trash and that shared albums/metadata remain correct.
- Only then permanently delete through Immich UI after a database/library backup.

Never directly delete library files or empty Trash globally without checking unrelated user data.

15. Detailed chronological lessons learned

Phase A — initial architecture

- Tailscale was chosen as the preferred secure remote-access method instead of public port forwarding.
- Immich mobile access should use a private reachable endpoint, not assume a UGREEN browser proxy is an API endpoint.
- Docker terminal access was identified as sufficient for running migration commands remotely.

Phase B — first import tests

- A reference ZIP was kept separately from each user's Takeout ZIPs.
- immich-go --dry-run was used first to understand counts and metadata behavior.
- --sync-albums=true was kept for every migrating user because each user owns some albums shared with others.
- The first actual failures came from trying to pause Immich jobs without admin permission; --pause-immich-jobs=false became the standard.

Phase C — first reconciler behavior

- The original plan compared SHA-1 values and attempted to attach existing assets.
- A major false-negative was caused by relying on the album GET response for membership; Immich did not return the full asset list there.
- Metadata search by albumIds fixed the membership check.
- A rollback journal was added so membership changes could be removed without deleting media.

Phase D — multi-user and album-member behavior

- The correct source key is required for an existing cross-user asset.
- The source user must be an album editor.
- Nested role responses (albumUsers[].user.id) caused misleading "member absent" checks until parsing was corrected.

- Automatic member sharing was added to the config and journal.

Phase E — large batches

- Large inventory refreshes were slow because users had tens of thousands of assets and SQLite replacement/index writes were expensive.
- The targeted refresh mode was introduced to refresh only the owner's recent assets after an external upload.
- Plans were split into smaller batches to reduce disk and operational risk.
- New journals were used for retries, while existing album membership prevented successful prior items from being reattached.

Phase F — ambiguous/fallback/external cases

- Filename+size matches were sometimes ambiguous. The user chose not to allow fallback and preferred fresh uploads for uncertain cases.
- Some owner uploads returned code 0 but were not found in the refreshed inventory. Immich's duplicate behavior and video processing delays were the main suspects.
- The practical solution was to inspect the immich-go log, search Immich by checksum, and manually attach or leave an already-correct album item rather than retrying indefinitely.
- A force_external_upload flag was added for cases where a fresh owner upload was an intentional policy decision.

Phase G — storage pressure

- The reference ZIPs, browser temporary files, and external staging directory consumed substantial space.
- The Immich library was on a different path and was much larger than /srv/migration alone suggested.
- Cleanup had to distinguish temporary migration files from Immich-managed originals/thumbnails/database data.
- The correct sequence became: finish/verify an album, preserve the journal, remove its reference/staging files, then start the next batch.

Phase H — remaining Large shared album album

- Immich showed the current server count items while the original Google Photos Large shared album album showed the source count.
- Because the album is large and Google Photos UI could not export it in one convenient operation, the proposed approach is two temporary albums.
- The two temporary ZIPs become two references for the same canonical Immich Large shared album album. Existing items should be skipped; missing external items should be matched/uploaded according to the normal policy.

16. Recommended Large shared album procedure

16.1 Create the two temporary albums in Google Photos UI

Use the original Large shared album album as the source.

- Create Large shared album - Part 1 and Large shared album - Part 2.
- Select a deterministic split, preferably by date or by contiguous visual order.
- Ensure every original item belongs to exactly one temporary album.
- Record the counts. The two counts must sum to exactly the source count.

- Do not delete the original Large shared album album.
- Include videos and any externally contributed items; do not select only photos.

Date-based splits are easier to audit. If Google Photos ordering is unclear, use batches with visible counts and keep screenshots/notes.

16.2 Download directly to the NAS

From NAS-hosted Chromium:

- Open Large shared album - Part 1.
- Download all to /downloads.
- Wait for the final .zip.part rename or CRC completion.
- Verify the ZIP with unzip -t.
- Repeat for Part 2.

Do not start the reconciler against a .zip.part file.

16.3 Use two references for the same Immich album

The config can contain:

```
{
  "name": "Large shared album",
  "owner": "User C",
  "reference": [
    "/srv/migration/reference/Large shared album - Part 1.zip",
    "/srv/migration/reference/Large shared album - Part 2.zip"
  ]
}
```

First run plan. Expect a large already_in_album count and inspect external_missing/ambiguous. If the two ZIPs overlap, the script's reference checksum de-duplication will prevent duplicate reference processing, but the split should still be corrected because exact coverage matters.

16.4 Apply conservatively

Because storage is tight:

- Plan using the existing inventory unless user imports changed.
- Apply existing matches first without --upload-missing if external candidates are numerous.
- Verify the album count.
- Apply external uploads only after confirming the remaining reference items are truly absent from all five users.
- Remove each temporary ZIP/staging directory only after final verification.

17. Useful command appendix

Check process state

```
ps -o pid,etime,time,%cpu,%mem,stat,cmd -p "$PID"
ps aux | grep -E 'google_album_reconciler|immich-go' | grep -v grep
```

Check a journal summary

```
jq '{status, warning, error, completed_at, failed_at}' \
  /srv/migration/reconciler-state/apply-YYYY-MM-DD.json
```

List upload failures

```
jq -r '
  .albums[] as $album |
  $album.upload_failures[]? |
  "\($album.name) | \(.reference) | owner=\(.user) | matches=\(.matches_under_owner) | immich-
go=\(.immich_go_returncode)"
' /srv/migration/reconciler-state/apply-YYYY-MM-DD.json
```

Find the last reconciler events

```
jq -r '
  .albums[] |
  .events[-10:][]? |
  "\(.at) | \(.outcome) | \(.reference) | \(.asset_id // "-")"
' /srv/migration/reconciler-state/apply-YYYY-MM-DD.json
```

Test the Immich API

```
SERVER='http://NAS_HOST:2283'

curl --connect-timeout 10 --max-time 30 -o /dev/null \
  -w 'HTTP %{http_code}\n' \
  -H "x-api-key: $IMMICH_USER_B_API_KEY" \
  -H 'Content-Type: application/json' \
  -X POST "$SERVER/api/search/metadata" \
  --data '{
    "page": 1,
    "size": 1,
    "withArchived": true,
    "withDeleted": false,
    "withExif": true,
    "withPeople": false,
    "withStacked": true,
    "order": "asc"
  }'
```

Search an album by name

```
curl -fsS \
  -H "x-api-key: $IMMICH_USER_C_API_KEY" \
  "$SERVER/api/albums" |
  jq -r '.[] | select(.albumName | test("Large shared album"; "i")) | [.albumName,.id,.ownerId]
| @tsv'
```

Split a plan by album count

Always preserve the original plan. Example:

```
PLAN=/srv/migration/reconciler-state/plan-all.json
MID=10

jq --argjson mid "$MID" '.albums |= .[:$mid]' \
  "$PLAN" > /srv/migration/reconciler-state/plan-batch-1.json

jq --argjson mid "$MID" '.albums |= .[$mid:]' \
  "$PLAN" > /srv/migration/reconciler-state/plan-batch-2.json
```

Review each extracted plan before applying. Do not split through a shell variable containing unvalidated text.

Validate the local copies before deployment

```
cmp -s outputs/google_album_reconciler.py \
  docker/test-migration/app/google_album_reconciler.py
python3 -m py_compile \
  outputs/google_album_reconciler.py \
  docker/test-migration/app/google_album_reconciler.py
```

18. Things not to do

- Do not expose Immich 2283, SSH 22, SMB, Docker, or UGOS admin ports to the public Internet just to make remote access work.
- Do not paste API keys into JSON config, scripts, screenshots, or chat.
- Do not use an API key value as the api_key_env config field.
- Do not use an album owner's key to add another user's asset unless the API explicitly allows it; use the source owner's editor key.
- Do not assume Google Takeout contains every item visible in a shared album.
- Do not apply plans with ambiguous files.
- Do not enable filename+size fallback globally without reviewing collision risk.
- Do not delete reference files while an import/reconciler/browser download is using them.
- Do not delete files directly from /srv/immich/library.
- Do not delete or rebuild SQLite while another plan/apply process is active.
- Do not expose PostgreSQL or connect to it through 127.0.0.1 from a separate container.
- Do not reuse a journal path for an unrelated retry.
- Do not conclude that immich-go failed solely from a small nonzero error count; inspect the log and server state.

19. Final operational checklist

Before each album:

- [ ] Storage checked.
- [ ] No other reconciler/import process is running.
- [ ] Reference ZIP exists, is complete, and passes unzip -t.
- [ ] Config album name and owner match Immich exactly.
- [ ] API keys resolve to the intended users.
- [ ] Canonical owner album exists.
- [ ] Required members are editors.
- [ ] Plan has no ambiguous files.
- [ ] External candidates have been reviewed.
- [ ] Current script version is deployed on the NAS.

After each album:

- [ ] Journal says completed, or all partial failures are understood.

- [ ] Immich album count is correct.
- [ ] Videos and special media were spot-checked.
- [ ] Ownership/timeline placement is correct.
- [ ] Members can view the album.
- [ ] Reference and staging cleanup is safe.
- [ ] Plan and journal are archived before cleanup.